Fase Groove Arranger — Report finale di audit

Data: 27 ottobre 2026 **Repository:** `wsx79/fasegroovearranger` (branch `main`) **Commit analizzato:** `fa29d23` — *"Hand the pointer back to the arranger when Studio closes"* **Tipo di lavoro:** revisione statica in **sola lettura**. Nessun file di codice è stato modificato, nessun commit è stato creato per questo report. **Perimetro:** tutto il codice applicativo sotto `Source/` (~11.000 righe, esclusa `ThirdParty/`), più `tests/`, `scripts/`, `.github/workflows/`.

INDICE

- 0. Sintesi
- 1. Bug aperto: l'area client della DAW è muta dopo la chiusura di STUDIO

1.1 Sintomo (confermato dall'utente)

1.2 Cosa è già stato provato

1.3 Fatti accertati leggendo il codice

1.4 Ipotesi, in ordine di probabilità

1.5 Test discriminanti (3 minuti, nessuna build)

1.6 Patch diagnostica pronta (da applicare solo per la misura)

1.7 Workaround per l'utente, nel frattempo

1.8 Nota sul fix `fa29d23`

- 2. Critici

C1 — `Cancel` su COLLECT & SAVE / RELINK: kill del thread, file orfani, lock in stato indefinito

C2 — Chiusura della finestra AI durante una generazione: `std::thread` non joinato su memoria liberata

- 3. Alti

A1 — Corsa sullo stato `Handoff` : `SampleSynth::configure()` legge un `MixState` già ritirato

A2 — `StudioWindow` : `selected()` non controllato, con `refresh()` saltato quando un campo ha il focus

A3 — L'autosave silenzia l'uscita audio e blocca la UI

A4 — Durante la registrazione Studio disabilita tutta la finestra principale

- 4. Medi

M1 — MIDI LEARN azzera la progressione al nono accordo

M2 — Drop di un file sulla timeline: sposta la testina e usa la traccia selezionata

M3 — La take viene chiusa e committata *prima* del prompt Salva/Scarta/Annulla

M4 — `requestClose()` restituisce `false` subito dopo aver aperto il file chooser

M5 — Limiti applicati dopo il parsing completo (JSON) e import MIDI senza tetto

M6 — Editor: checkpoint vuoto sul click a vuoto nella corsia velocity; gomma solo al mouseDown

M7 — Percorsi relativi dei sample senza controllo di `..`

M8 — Finestra AI: elenco tracce indicizzato per posizione, non risincronizzato

- 5. Bassi

B1 — Export MIDI: collisione di canale con molte tracce manuali

B2 — APPLY della Voicing Coach usa l'ultima voce ascoltata

B3 — Relink per solo nome file sui progetti legacy

B4 — Test: attesa senza timeout che può appendere la CI

B5 — Test: asserzione debole e soglia temporale

B6 — CI: verifica audio solo su Linux, strumenti di sincronizzazione non usati

- 6. Cosa non ho potuto verificare
- 7. Ordine di intervento consigliato
- 8. Appendice — metodo

0. Sintesi

Gravità	Numero	Significato
Critico	2	Blocco dell'app o memoria liberata mentre è ancora in uso
Alto	4	Crash/glitch possibili in uso normale, o stato dell'app corrotto
Medio	8	Comportamento sbagliato visibile all'utente, workaround esistenti
Basso	6	Fragilità, test/CI poco affidabili, casi limite
Totale	**20**	

I cinque interventi con il rapporto rischio/beneficio migliore

1. **C1** — `Cancel` su COLLECT & SAVE / RELINK può terminare il thread a forza (`TerminateThread`) lasciando file, handle e lock in stato indefinito (`StudioWindow.cpp:139/199/401`).
2. **C2** — chiusura della finestra AI durante una generazione: il watcher HTTP sopravvive al thread terminato e usa memoria liberata (`AiComposer.cpp:132–134`).
3. **A1** — corsa `Handoff` : l'audio thread legge `*parameters` di uno stato già passato a `retired` , che il message thread può liberare nello stesso istante (`Sampler.cpp:119` + `MainComponent.cpp:1174`).
4. **A4** — durante la registrazione Studio disabilita **tutto** il `MainComponent` 15 volte al secondo e, a finestra chiusa, il timer si ferma lasciando lo stato congelato (`StudioWindow.cpp:373/402`). È anche il primo indiziato del bug aperto (§1).
5. **M2** — trascinare un file sulla timeline sposta la testina e carica il campione sulla traccia **selezionata**, non su quella sotto il puntatore (`MainComponent.cpp:1337–1341`).

Stato del bug segnalato dall'utente (area client muta dopo la chiusura di STUDIO): ancora aperto. Il fix `fa29d23` (rilascio della mouse capture Win32 + `toFront` + `triggerFakeMove`) non ha cambiato il sintomo: la causa non è il percorso di input a livello Win32. L'analisi aggiornata, le ipotesi ordinate e un test da 3 minuti sono nel §1.

---

1. Bug aperto: l'area client della DAW è muta dopo la chiusura di STUDIO

1.1 Sintomo (confermato dall'utente)

- Dopo aver chiuso la finestra **STUDIO**, il contenuto della finestra principale non reagisce: niente hover sui pulsanti, il primo click non produce effetti.
- La **barra del titolo** (nativa) resta trascinabile subito.
- **Un click qualsiasi** riporta tutto alla normalità.
- Succede solo con STUDIO: non con MIXER, non con la finestra dello strumento, non con l'editor VST.

1.2 Cosa è già stato provato

Il commit `fa29d23` ha aggiunto in `StudioWindow::closeButtonPressed()` il rilascio di `GetCapture()` (anche in differita con `MessageManager::callAsync`), `setVisible(false)` , `toFront(true)` sulla finestra principale e `triggerFakeMove()` .

Non ha funzionato. Questo **esclude** che l'input resti dirottato a livello Win32 (capture o finestra nascosta che riceve i messaggi): se fosse così, il rilascio della capture avrebbe risolto, e comunque il titolo nativo non sarebbe stato trascinabile.

1.3 Fatti accertati leggendo il codice

1. **MIXER, strumento e finestre VST usano la stessa ricetta di finestra** (`setUsingNativeTitleBar(true)` + `setAlwaysOnTop(true)` + `setVisible(false)`), ma non producono il difetto: `MixerRoutingWindow.cpp:98–101` , `MainComponent.cpp:576/599` . Stile finestra, z-order e attivazione sono quindi esclusi come causa.
2. **Studio è l'unica finestra secondaria che modifica lo stato globale del proprietario**, 15 volte al secondo: `StudioWindow.cpp:373`

`owner.setEnabled(!owner.audio.isRecording() && !audioRecording && !busy);`

1. **A finestra chiusa il suo timer si spegne quasi subito** (`StudioWindow.cpp:402`): `if(!isShowing() && ...) return;` — quindi l'ultimo stato scritto in `owner` **resta congelato** finché Studio non viene riaperto.
2. `endSession()` ripristina solo `owner.setEnabled(!busy)` (`StudioWindow.cpp:265`): se al momento della chiusura `busy == true` (task COLLECT & SAVE / RELINK ancora vivo, `StudioWindow.cpp:401`) la finestra principale resta **disabilitata**.
3. `MainComponent` disabilitato significa: niente click né tastiera sul contenuto, **ma la barra del titolo nativa continua a funzionare**. È esattamente la geometria del sintomo.
4. Lo stesso timer di Studio decide se ridisegnarsi guardando lo stato dei bottoni del mouse (`StudioWindow.cpp:404` ,

`MainComponent.cpp:1166`): uno stato mouse "bloccato" congelerebbe anche l'aggiornamento della finestra.

1.4 Ipotesi, in ordine di probabilità

#	Ipotesi	Spiegazione	Punti a favore	Punti deboli
H1	`MainComponent` **lasciato disabilitato**	`owner.setEnabled(false)` scritto durante registrazione o task media resta congelato perché il timer di Studio non gira più	Unica scrittura globale di Studio; spiega hover morto, click assorbito, titolo nativo attivo	Non spiega da solo perché *un* click riattivi tutto
H2	**Stato del mouse JUCE bloccato in "trascinamento"**	La finestra viene nascosta mentre un drag è in volo: `MouseInputSource` crede il tasto ancora premuto → i movimenti diventano `mouseDrag` verso un componente sparito (niente hover) e il click successivo non genera un nuovo `mouseDown` (click "mangiato")	Spiega **tutti** i sintomi, incluso il "primo click perso e poi tutto ok"	Serve una sorgente di drag attiva alla chiusura
H3	**Blocco modale residuo**	Un `ThreadWithProgressWindow` (task media) o un file chooser resta registrato come modale: JUCE consuma il primo click per portarlo in primo piano	Il "click che sblocca" è letteralmente il comportamento di `isCurrentlyBlockedByAnotherModalComponent()`	Se il modale restasse registrato, ogni click verrebbe consumato, non solo il primo
H4	**Repaint affamato**	Il timer principale ridisegna solo se qualcosa si muove (`MainComponent.cpp:1166-1172`); il primo click imposta `mouseDown`, quindi riparte il ridisegno completo	Spiega "hover morto" e "il click riaccende tutto"	Non spiega perché il click non produca il suo effetto

1.5 Test discriminanti (3 minuti, nessuna build)

Con il difetto attivo, appena chiusa la finestra STUDIO:

1. **Premi SPACE.** Se parte l'audio → tastiera viva: il problema è solo nel percorso del mouse (H2/H4). Se non succede nulla → il contenuto è **disabilitato** o bloccato (H1/H3).
2. **Guarda la barra del titolo**: grigia/inattiva = finestra non attiva; colorata = finestra attiva (esclude i problemi di attivazione).
3. **Muovi il mouse sui bordi del pannello dell'editor**: se compare il cursore di ridimensionamento (`LeftRightResize`), le mosse **arrivano** e manca solo il ridisegno → H4.
4. **Prova a trascinare** un clip o un box valore senza click preliminare: se il trascinamento parte, il tasto è considerato già premuto → H2.
5. **Ripeti chiudendo Studio in tre modi**: X nativa, `Esc`, chiusura mentre una registrazione è in corso. Se il difetto compare solo in un caso, la sorgente è quello stato.

1.6 Patch diagnostica pronta (da applicare solo per la misura)

Da inserire in `StudioWindow::closeButtonPressed()`, dopo `setVisible(false)`, e in un `mouseMove` temporaneo di

`MainComponent`, scrivendo su `%TEMP%\fga-mouse.log`:

```
auto dump=[this](const char* when){
    auto& d=juce::Desktop::getInstance();
    auto* peer=owner.getPeer();
    juce::String s;
    s << when
      << " enabled=" << (int)owner.isEnabled()
      << " modal=" << (void*)juce::Component::getCurrentlyModalComponent()
      << " mouseDragging=" << (int)d.getMainMouseSource().isDragging()
      << " underMouse=" << (void*)d.getCurrentComponentUnderMouse()
      << " mods=" << juce::String::toHexString(juce::ModifierKeys::currentModifiers.getRawFlags())
      << " peerVisible=" << (int)(peer&&peer->isVisible())
      << " focused=" << (int)(peer&&peer->isFocused());
    #if JUCE_WINDOWS
    s << " capture=" << (void*)::GetCapture() << " fg=" << (int)(::GetForegroundWindow()==(HWND)peer->getNativeHandl
    #endif
    juce::FileLogger::createDateStampedLogger("fga", "mouse-", ".log", "")->logMessage(s);
};
dump("close-before"); juce::MessageManager::callAsync([this,dump]{if(panel)dump("close-after");});
```

La prima riga che differisce tra un caso sano e uno rotto identifica l'ipotesi vincente: `enabled=0` → H1; `mouseDragging=1` → H2; `modal!=0` → H3; tutto a posto ma hover assente → H4.

1.7 Workaround per l'utente, nel frattempo

Un click sull'area vuota della timeline dopo la chiusura di Studio. Il difetto è solo percettivo finché non viene identificato: nessun dato viene perso.

1.8 Nota sul fix `fa29d23`

Il rilascio della capture è corretto in sé (una finestra nascosta non deve trattenere il puntatore) e non introduce regressioni, ma **non è la causa**. Va tenuto se si vuole, non è la correzione.

---

2. Critici

C1 — `Cancel` su COLLECT & SAVE / RELINK: kill del thread, file orfani, lock in stato indefinito

File: `Source/StudioWindow.cpp:139`, `:199`, `:401` · `Source/StudioIO.cpp:14-26`, `:57-68` **Confidenza:** alta sul blocco, media sull'esito del kill (dipende dal punto di interruzione)

`MediaTask` è un `ThreadWithProgressWindow(..., true, true)` con timeout di cancellazione 10000 ms, ma il lavoro pesante non è interrompibile: `copyMedia()` non riceve né consulta mai il predicato di cancellazione; `collectProjectMedia()` lo controlla solo **tra un file e l'altro** (`StudioIO.cpp:61`).

Alla scadenza JUCE termina il thread a forza (`TerminateThread`) e poi chiama `threadComplete(true)`. Il thread terminato non esegue distruttori: il `juce::TemporaryFile` di `copyMedia` non viene cancellato, gli stream restano aperti (handle Windows persi fino alla chiusura del processo), la pulizia `for(auto& file:created)file.deleteFile();` (`StudioIO.cpp:68`) non gira, e `TerminateThread` lascia lock/CRT in stato non specificato.

Trigger: Studio → MEDIA → COLLECT & SAVE su un progetto con un sample grande o su disco lento, poi `Cancel`. **Impatto:** UI

congelata fino a 10 s ("non risponde"), copie orfane nella cartella `.media`, possibile progetto su disco "collected" mentre quello in memoria ha ancora i path vecchi. Il `.fga` esistente non si corrompe (usa `TemporaryFile`). **Correzione minima:** passare `cancelled` dentro `copyMedia` e copiare/hashare a blocchi controllando tra un blocco e l'altro; in alternativa rimuovere `Cancel` e usare una finestra non annullabile. Non usare il kill a 10 s come unica via d'uscita.

C2 — Chiusura della finestra AI durante una generazione: `std::thread` non joinato su memoria liberata

File: `Source/AiComposerWindow.cpp:13`, `:71`, `:165` · `Source/AiComposer.cpp:132-134` **Confidenza:** alta sul blocco, media sulla UAF (richiede che il worker non esca entro 5 s)

```
// AiComposerWindow.cpp:13
~CompositionJob() override { signalThreadShouldExit(); stopThread(5000); }
```

`cancelJob()` è chiamata da Cancel, da `closeButtonPressed()` e dal distruttore del pannello; se il worker è dentro `stream.connect()` (timeout fino a 900 s) non esce in 5 s e JUCE lo termina. La cancellazione HTTP vive in un watcher con distruttore di stack:

```
// AiComposer.cpp:133-134
std::thread watcher([&]{ while(!finished.load()){ if(cancelled()||...){ stream.cancel(); return; } } });
struct Finish { ~Finish(){ done = true; thread.join(); } } finish{finished, watcher};
```

Con il thread terminato `~Finish` non gira: `watcher` continua a leggere `finished`, `timedOut`, `stream` (frame dello stack del thread morto) e `cancelled` (cattura `this` = `CompositionJob` già distrutto).

Trigger: chiudere "Create with AI" o premere Cancel su endpoint lento/irraggiungibile; oppure cambiare provider mentre il check Ollama è in volo. **Impatto:** freeze fino a 5 s a ogni cancel; nel ramo di kill, use-after-free non deterministica e handle HTTP non chiusi. **Correzione minima:** non distruggere il job dal callback UI — `signalThreadShouldExit()` e deallocazione dopo il ritorno di `run()` (contenitore "zombie" svuotato dal message thread); rendere il watcher joinabile da chi sopravvive al worker.

---

3. Alti

A1 — Corsa sullo stato `Handoff`: `SampleSynth::configure()` legge un `MixState` già ritirato

File: `Source/Sampler.cpp:119` · `Source/AudioEngine.cpp:106`, `:108`, `:116` · `Source/AudioEngine.h:47-53` · `Source/MainComponent.cpp:1174` **Confidenza:** alta sul meccanismo, probabilità bassa-ma-reale

```
// AudioEngine.h:53
T* adopt(){ if(retired.load()!=nullptr)return nullptr; T* next=pending.exchange(nullptr);
           if(next==nullptr)return nullptr; retired.store(current); current=next; return next; }
```

L'audio thread, dentro `adoptState()`, pubblica il vecchio stato in `retired` e subito dopo chiama `applyMix(*next)`, che a sua volta chiama `sampler->configure(...)`. `configure()` apre con:

```
// Sampler.cpp:119
const auto& old = *parameters;   // parameters punta ancora al MixState PRECEDENTE
```

quel `parameters` punta dentro lo stato appena finito in `retired`. Il message thread libera `retired` in `collectGarbage()` → `mixes.collect()` → `delete retired.exchange(nullptr)`, e `collectGarbage()` è chiamato dal timer della UI **senza** `EngineControlLock` (`MainComponent.cpp:1174`).

Finestra di esposizione: la durata di `applyMix` (microsecondi) per ogni modifica di mix, con il timer a 30 Hz. Rara, ma è una use-after-free sul thread audio: crash difficile da associare alla causa. **Correzione minima:** memorizzare i valori che servono al riavvio (`mode` , `sync` , `reverse` , `start` , `end` , `sliceCount` , `slices`) nella voce dell'instrument quando lo stato viene adottato, oppure rimandare la `configure()` al controllo successivo e leggere i dati dal **nuovo** stato. In alternativa, non liberare `retired` finché l'audio thread non ha completato l'adozione (contatore di epoch).

A2 — `StudioWindow` : `selected()` non controllato, con `refresh()` saltato quando un campo ha il focus

File: `Source/StudioWindow.cpp:339` , `:351` , `:402` , `:404` · `Source/MainComponent.cpp:1238` **Confidenza:** media (la finestra di esposizione è stretta ma reale)

```
// StudioWindow.cpp:339
Track& selected(){ return owner.project.tracks[(size_t)track]; }
```

L'unico clamp è in `refresh()` (`:351`). Ma il timer evita `refresh()` quando il focus è in un `TextEditor` (`:404`), e i box dei valori degli slider **sono** `TextEditor` : con il focus lì il clamp non gira più. Il commit del testo passa proprio da `selected()` (`:344–345`), così come `wave.onRegion` / `onMarker` (`:162`), `sampleMode.onChange` (`:168`), LOAD SAMPLE (`:163`) e `MainComponent::loadSampleFile` (`:1238` , nessun controllo).

Trigger: con Studio aperto, eliminare una traccia MIDI (o undo/redo di una cancellazione) e agire su un controllo Studio prima del successivo `refresh()` completo; illimitato se il focus è in un box di valore. **Impatto:** lettura/scrittura fuori dai limiti di `std::vector<Track>` — crash o scrittura su heap adiacente; `loadSampleFile` carica il campione sulla traccia sbagliata. **Correzione minima:** `Track* selected()` che restituisce `nullptr` fuori range e uscita anticipata dai callback; stesso controllo in testa a `loadSampleFile` .

A3 — L'autosave silenzia l'uscita audio e blocca la UI

File: `Source/MainComponent.cpp:1175` · `Source/AudioEngine.cpp:458–460` **Confidenza:** alta (documentata dal commento stesso)

```
// AudioEngine.cpp:459–460
// The callback never waits on the control mutex; it emits silence during this short snapshot.
void AudioEngine::capturePluginStates(Project& p){ ... const EngineControlLock lock(*this); ... }
```

`timerCallback()` chiama `capturePluginStates()` ogni 1200 tick **anche durante la riproduzione**, poi serializza il progetto e lo scrive su disco sul message thread (`saveProject`). Effetto: buco di silenzio udibile (e un possibile click) mentre si suona, più uno stallo della UI proporzionale al numero di plugin e alla dimensione del progetto. **Correzione minima:** autosalvare solo a transport fermo oppure con `capturePluginStates` eseguito su uno stato congelato (plugin non realtime / copia offline, come fa già l'export).

A4 — Durante la registrazione Studio disabilita tutta la finestra principale

File: `Source/StudioWindow.cpp:373` , `:265` , `:402` **Confidenza:** alta

```
// StudioWindow.cpp:373
void refreshAudioControls(){ ... owner.setEnabled(!owner.audio.isRecording() && !audioRecording && !busy); ... }
```

Con una take in corso il **trasporto della finestra principale** (play/stop, timeline, pulsanti) è disabilitato: se Studio finisce dietro

un'altra finestra non c'è modo di fermare la registrazione dalla DAW. A finestra chiusa il timer esce subito (`:402`), quindi l'ultimo stato scritto resta congelato; `endSession()` lo ripristina solo con `!busy` (`:265`), e un task media ancora attivo lo lascia disabilitato. **Impegno:** il pulsante STOP della finestra principale deve restare utilizzabile; disabilitare solo i controlli che porterebbero a uno stato incoerente. **Nota:** questo è il primo indiziato del bug aperto (§1, H1) e va toccato insieme alla diagnosi.

---

4. Medi

M1 — MIDI LEARN azzera la progressione al nono accordo

File: `Source/PianoTools.cpp:72`

```
s.chordCount = slot == 0 ? 1 : std::max(slot + 1, s.chordCount);
...
slot = (slot + 1) % 8;
```

`slot` torna a 0 dopo l'ottavo accordo; il nono viene scritto nello slot 0 e forza `chordCount = 1` : la progressione imparata viene troncata al primo accordo. **Fix:** contatore monotonico separato dall'indice dello slot circolare.

M2 — Drop di un file sulla timeline: sposta la testina e usa la traccia selezionata

File: `Source/MainComponent.cpp:1337-1341`

```
if(point.x >= view.getX()+headerWidth){
    seekTimeline(std::max(0., (point.x-headerWidth)*4./arrangement.pixelsPerBar));
    selectClip(track, project.clipAtBeat(track, timelinePosition()));
} else select(track, selectedSection);
...
if(file.hasFileExtension("mid;midi")) importFile(file); else loadSampleFile(selectedTrack, file);
```

Il solo drop **cambia la posizione del transport** (e con `MIDI` importa nella sezione della testina, non in quella sotto il puntatore) e carica il campione in `selectedTrack` , ignorando la traccia su cui è stato rilasciato. **Fix:** nessun `seekTimeline` sul drop; usare la traccia della riga sotto il puntatore e la sezione della posizione di drop.

M3 — La take viene chiusa e committata *prima* del prompt Salva/Scarta/Annulla

File: `Source/MainComponent.cpp:1224`

```
bool MainComponent::requestClose(){
    if((audio.isRecording()||audio.isAudioRecording()) && studioWindow) studioWindow->closeButtonPressed();
    if(!dirty) return true;
    ... showYesNoCancelBox(...)
```

Chiudendo l'app durante una registrazione, la take viene finalizzata e inserita in Undo prima che l'utente scelga "Annulla": la chiusura viene annullata ma la registrazione è già stata fermata e committata. **Fix:** chiedere prima, e finalizzare la take solo se la scelta non è "Annulla".

M4 — `requestClose()` restituisce `false` subito dopo aver aperto il file chooser

File: `Source/MainComponent.cpp:1228` , `:1234` · `Source/Menus.cpp:108-110`

```
if(loadingPlugin || projectFile == juce::File()){ save(true); return false; }
```

`save(true)` è asincrono (file chooser), ma la funzione dichiara subito "non chiudere". I chiamanti interpretano il `false` come "l'utente ha annullato" e **abortiscono in silenzio**: `open()` (`:1234`), nuovo progetto, e "Recupera autosave" (`Menus.cpp:108`). **Conseguenza verificata:** con un progetto nuovo non ancora salvato, *Impostazioni audio → Recupera autosave* apre il dialogo "Salva con nome" e poi non recupera nulla, senza messaggi. **Fix:** far restituire `requestClose()` uno stato a tre valori (`close` , `cancelled` , `pending`) e riprendere l'azione nel callback del chooser.

M5 — Limiti applicati dopo il parsing completo (JSON) e import MIDI senza tetto

File: `Source/ProjectIO.cpp:208` , `:188` , `:212`

- `loadProject()` chiama `juce::JSON::parse(file.loadFileAsString(), json)` **prima** di `projectFromJson` : il limite di 2.000.000 note (`:188`) e i limiti di struttura si applicano a parsing già avvenuto. Un `.fga` da 256 MB con array profondamente annidati può esaurire la memoria o lo stack prima del rifiuto.
- `importMidi()` non ha alcun tetto sulle note (`:212`); un MIDI molto denso produce un progetto che **non si riapre** (rifiutato dal limite in `:188`).

Fix: tetto sul numero di elementi e sulla profondità prima/durante il parsing; applicare lo stesso limite a `importMidi` con messaggio esplicito.

M6 — Editor: checkpoint vuoto sul click a vuoto nella corsia velocity; gomma solo al mouseDown

File: `Source/MainComponent.cpp:487` , `:490`

```
if(e.y>=velocityTop()){ owner.checkpoint(); editingVelocity=true; editVelocity(e.position); return; }
```

`editVelocity()` esce senza modifiche se non c'è una nota vicina, ma il checkpoint (voce di Undo) è già stato creato: un click a vuoto nella corsia velocity sporca la cronologia e riapre un progetto identico. Inoltre con lo strumento gomma (`tool==2`) la cancellazione avviene solo nel `mouseDown` su una nota: trascinare sopra più note non le cancella. **Fix:** checkpoint dopo aver verificato che ci sia una nota; cancellazione anche in `mouseDrag` .

M7 — Percorsi relativi dei sample senza controllo di `..`

File: `Source/StudioIO.cpp:54` , `:80`

```
return project==juce::File() ? juce::File::getCurrentWorkingDirectory().getChildFile(text)
                             : project.getSiblingFile(text.replaceCharacter('\\','/'));
```

Un `.fga` (o un `.fgs`) costruito a mano con `instrumentPath: "../../..."` fa leggere e caricare come sample un file arbitrario del disco. `loadSamplerPreset()` rifiuta i path assoluti ma non i `..` . **Impatto:** limitato ai file aperti dall'utente, ma è una via di fuga dalla cartella del progetto (e i percorsi finiscono nei progetti "collected"). **Fix:** rifiutare i path che, normalizzati, non restano figli della cartella del progetto.

M8 — Finestra AI: elenco tracce indicizzato per posizione, non risincronizzato

File: `Source/AiComposerWindow.cpp:67-70` , `:154` · `Source/AiComposer.cpp:83`

`refreshTracks()` è chiamata solo dal costruttore e da `show()`; la composizione invia gli **indici dei widget**, e la validazione lato motore controlla solo range e lock. Se una traccia MIDI viene cancellata o bloccata con la finestra aperta, gli indici slittano e le note finiscono su strumenti adiacenti; una traccia bloccata dopo l'apertura viene rifiutata solo **dopo** la chiamata HTTP. **Fix:** ricostruire i toggle all'inizio di `start()` e validare lock/indici prima di inviare la richiesta.

---

5. Bassi

B1 — Export MIDI: collisione di canale con molte tracce manuali

`Source/core/Midi.cpp:35-36` — i canali manuali sono `(5+k)%15` e poi corretti se `>=9`; con 12 o più tracce manuali si torna su canali già usati (es. `%15` → 1). Note di tracce diverse sullo stesso canale. **Fix:** assegnazione esplicita e verifica di unicità prima di scrivere.

B2 — APPLY della Voicing Coach usa l'ultima voce ascoltata

`Source/PianoTools.cpp:55` — `p.settings.voicing = chosenVoice>=0 ? chosenVoice : a.getSelectedItemIndex();`. Dopo un ascolto di B, APPLY applica B anche se nella lista è selezionata A. Da confermare come scelta voluta.

B3 — Relink per solo nome file sui progetti legacy

`Source/StudioIO.cpp:74-76` — con `sampleHash` vuoto il relink accetta un file con lo stesso nome anche se il contenuto è diverso; con più candidati e hash presente sceglie il primo. Comportamento voluto per i progetti vecchi, ma va segnalato all'utente ("contenuto non verificato").

B4 — Test: attesa senza timeout che può appendere la CI

`tests/EngineAuditTests.cpp:49` — `engine.mutex.enter(); std::thread independent(callback); independent.join();`. Se il callback regredisse e attendesse il mutex, `join()` non ritorna: la CI resta appesa invece di fallire. **Fix:** `wait_for` con timeout e `requireAudit`.

B5 — Test: asserzione debole e soglia temporale

- `tests/CoreTests.cpp:266`: `check(chordNames(...).size()==4 || p.settings.chordCount!=4, "harmony")` passa comunque quando `chordCount != 4`, indipendentemente dal risultato.
- `tests/SampleLibraryTests.cpp:75`: ordinare 20.000 file entro 2000 ms è una soglia dipendente dalla macchina (flake su runner lenti).

B6 — CI: verifica audio solo su Linux, strumenti di sincronizzazione non usati

`.github/workflows/build.yml:77-95` — `--smoke-test`, `verify-audio.py`, `--host-smoke-test` e `--club-audit` girano solo su `ubuntu-24.04`; macOS e Windows si fermano a build + `ctest`. Le differenze di rendering/plugin specifiche di piattaforma non sono coperte. Inoltre `scripts/sync-engines.py --check` esiste ma non è invocato da nessun job (la coerenza dei moduli è verificata dal lock file, non dai motori vendored). **Fix:** eseguire almeno `verify-audio.py` e `--club-audit` anche su Windows (con device offline) e aggiungere il job `sync-engines --check` in CI.

---

6. Cosa non ho potuto verificare

- **Nessuna build**: sulla macchina dell'analisi non ci sono `cmake`, MSVC, `g++`, `clang` né `gh`. Non ho compilato, non ho eseguito

i test, non ho riprodotto il bug.

- Il bug del §1 resta **non riprodotto**: le ipotesi sono ordinate per compatibilità con i sintomi, non confermate sperimentalmente.
- Le prestazioni (latenza dell'autosave, durata del sort di 20.000 file) sono stimate dal codice, non misurate.
- Due segnalazioni raccolte durante l'analisi sono state **scartate** dopo verifica: il relink **non** salta il controllo di hash quando l'hash è presente (`StudioIO.cpp:75`), e la take MIDI non si perde chiudendo Studio (`endSession()` / `~Panel` chiamano `finish()`).

---

7. Ordine di intervento consigliato

Ordine	Intervento	Perché ora
1	A4 + diagnosi §1.6	Sblocca l'uso della DAW e chiude il bug aperto con un dato, non con un'altra ipotesi
2	C1, C2	Eliminano i due percorsi che possono lasciare stato indefinito nel processo
3	A1	UAF sul thread audio: una volta corretta, spariscono crash non riproducibili
4	M3, M4, M5	Perdita/abbandono silenzioso di lavoro dell'utente
5	M1, M2, M6, M8	Difetti d'uso quotidiano, correzioni piccole e isolate
6	A2, A3, M7	Robustezza e igiene
7	B1-B6	Test e CI: rendono visibili le regressioni future

---

8. Appendice — metodo

- Lettura integrale di `Source/*.cpp` , `Source/*.h` , `Source/core/*` (esclusi `ThirdParty/`), `tests/*` , `scripts/*` , `.github/workflows/build.yml` .
- Ogni finding è stato ricontrollato sul sorgente e annotato con file:riga; le voci non riconfermate sono state scartate (§6).
- Comportamento di JUCE verificato sui sorgenti 8.0.9 (versione fissata in `CMakeLists.txt:66`): `ThreadWithProgressWindow` , `Thread` su Windows, `Handoff` / `MouseInputSource` , `AlertWindow` .
- Nessun comando ha modificato il repository: `git status` resta pulito rispetto a `fa29d23` .